

Partea I:

Lecture 6: Java RMI (Remote Method Invocation)

1. RMI Overview

- **Purpose:** Allows a Java program on one JVM (client) to invoke methods on an object residing in another JVM (server) as if it were local.
 - **Key Benefit:** Hides networking details (sockets, serialization) behind familiar method calls.
-

2. RMI Architecture s Components

1. RMI Registry

- A simple name service (running by default on port 1099) where servers **bind** remote objects under a logical name.
- Clients **lookup** that name to obtain a stub reference.

2. Stub s Skeleton

- **Stub** (client-side proxy):
 - Implements the remote interface.
 - Marshals (serializes) method parameters over the network.
- **Skeleton** (server-side dispatcher):
 - Unmarshals incoming calls and invokes the actual implementation.
 - Marshals the return value back to the stub.
- **Note:** As of Java 5, skeletons are no longer generated; the runtime dispatches directly to server objects via dynamic proxies.

3. JRMP Protocol

- Java's proprietary "Java Remote Method Protocol" used by RMI.

4. Security Manager s Policy

- A `RMISecurityManager` (or custom `SecurityManager`) must be set to enforce a security policy (`java.policy`) that governs class downloading and access rights.
-

3. RMI Development Steps

Server-Side

1. Define Remote Interface

```
public interface SampleServerInterface extends Remote {  
    int sum(int a, int b) throws RemoteException;  
}
```

2. Implement Remote Object

- Extend UnicastRemoteObject and implement the interface.
- Example constructor must throw RemoteException.

```
public class SampleServerImpl extends UnicastRemoteObject  
    implements SampleServerInterface {  
    public SampleServerImpl() throws RemoteException { super(); }  
    public int sum(int a,int b) throws RemoteException {  
        return a + b;  
    }  
}
```

``` :contentReference[oaicite:5]{index=5}

##### 3. Write Server Main

- Set security manager.
- Instantiate implementation and bind to registry:

```
System.setSecurityManager(new RMISecurityManager());
SampleServerImpl server = new SampleServerImpl();
Naming.rebind("rmi://localhost:1099/SAMPLE-SERVER", server);
``` :contentReference[oaicite:6]{index=6}
```

4. Compile s Generate Stubs

- javac the classes.
- Run rmic on the implementation to generate stub classes (unless using dynamic proxies).

5. Start Services

- Launch rmiregistry.
- Run the server main class.

Client-Side

1. Set Security Manager

2. Lookup Remote Object

```
System.setSecurityManager(new RMISecurityManager());
```

```
SampleServerInterface stub =
```

```
(SampleServerInterface) Naming.lookup("rmi://localhost:1099/SAMPLE-SERVER");
```

3. Invoke Remote Methods

```
int result = stub.sum(1, 2);
```

```
System.out.println("1 + 2 = " + result);
```

```
``` :contentReference[oaicite:7]{index=7}
```

---

## 4. Gotchas & Minor Details

- **RemoteException** must be declared on every remote method.
- **Serializable parameters/return types** only. All objects passed must implement `java.io.Serializable`.
- **Default RMI port 1099**: can be changed but client URL must match.
- **Dynamic Stubs**: Modern JVMs generate stubs at runtime—no `rmic` needed (but understanding `rmic` is still useful).
- **Firewall/Networking**: Ensure that both registry and remote object ports are reachable; consider using a custom socket factory for SSL or firewalled environments.
- **Class Loading**: By default, client must have access to the remote interface and stub classes; dynamic class downloading requires codebase settings in the policy file.

---

## 5. Exam-Focus Points

- **Definitions:** RMI, stub, skeleton, marshalling/unmarshalling, JRMP.
- **Use-Cases:** Simple distributed services where Java-to-Java calls are needed (e.g., in JEE/EJB).
- **Code Analysis:** Be ready to trace what happens when a client calls `stub.method()`: lookup → stub marshals args → skeleton unmarshals → server executes → result marshaled back.
- **Configuration Details:** Security manager, registry URL, default ports, exception handling.

## Lecture 8: SOA vs. REST s HTTP 1 vs. 2; Open MPI vs. OpenMP; IaaS Cloud s Open-MPI on AWS EC2

---

### 1. SOA vs. REST

#### a. Service-Oriented Architecture (SOA / WS-\*)

- **Definition:** An architectural style where discrete services communicate via standardized contracts (typically SOAP over HTTP, plus WSDL for interface description) .
- **Characteristics:**
  - Rigid contract ("big WSDL"): operations, message schemas explicitly defined.
  - Often heavy XML payloads, WS-\* standards (security, transactions).
  - Suited for enterprise scenarios requiring advanced QoS (security, reliability).

#### b. Representational State Transfer (REST)

- **Definition:** An architectural style exploiting the uniform interface of HTTP to treat "everything" as a resource, identified by URIs and manipulated via standard verbs (GET, POST, PUT, DELETE) .
- **Characteristics:**
  - **Stateless** interactions: each request contains all context.
  - **Lightweight representations:** JSON (preferred), XML, even plain text.
  - **Hypermedia** (HATEOAS) – clients discover actions via links.
  - Emphasizes resource modeling (nouns) over operations (verbs).
- **Gotcha:** Don't confuse "RESTful" with RPC over HTTP; true REST leverages HTTP semantics and status codes (e.g. 404 for not found, 201 for created) .

---

## 2. HTTP 1.1 vs. HTTP 2.0

Feature	HTTP 1.1	HTTP 2.0
<b>Framing</b>	Text-based	Binary frames (more compact, easier parsing)
<b>Multiplexing</b>	One request per TCP connection (requiring sharding)	Multiple streams per TCP connection; no head-of-line blocking
<b>Header Compression</b>	None	HPACK compression reduces overhead
<b>Server Push</b>	N/A	Server can proactively send resources into cache
<b>Connection Setup</b>	Separate TLS handshake per connection	Uses ALPN extension to negotiate HTTP/2 during TLS handshake
<b>Performance Optimizations</b>	Domain sharding, asset concatenation	No longer needed; single connection handles parallelism
<b>Requirement</b>	Can run over plain TCP	Must run over TLS (encrypted)

### Minor Details & Gotchas

- **Browser support:** Nearly universal for HTTP 2, but only over HTTPS; fallback to 1.1 otherwise .
- **Server Push misuse:** Pushing large assets can **hurt** performance if clients already cache them.
- **SPDY → HTTP 2:** Google's SPDY protocol was the precursor; many ideas (multiplexing, header compression) carried over .
- **HTTP 3:** Based on QUIC (UDP-based), in draft as of 2025—expect future migration.

---

### 3. Open MPI vs. OpenMP

Aspect	Open MPI (Message Passing)	OpenMP (Shared-Memory Threads)
Model	Distributed-memory; <b>each process</b> has its own heap; communicate via MPI_Send/MPI_Recv	Shared-memory; <b>threads</b> within one process share address space; parallel regions via pragmas
Deployment	Multi-node clusters (e.g., AWS EC2, HPC interconnect)	Single node / SMP machines (multi-core CPUs)
Synchronization	Explicit message primitives; can use collective ops (MPI_Bcast, MPI_Reduce)	Implicit barrier at end of #pragma omp for; explicit barriers available
Ease of Use	Steeper learning curve (must partition data, handle ranks)	Easier for loop-level parallelism (just add pragmas)
Scalability	Scales across nodes, networks; ideal for HPC clusters	Limited by node's shared memory; generally up to dozens of cores
Overhead	Network latency; message-copying	Thread-creation overhead; potential cache coherency issues

#### OpenMP Gotchas Details

- **Race conditions:** Variables default to shared—must declare private/reduction for safety .
- **Scheduling:** schedule(static[,chunk]), dynamic, guided, runtime, auto control workload distribution; wrong choice can cause load imbalance .
- **I/O:** Not covered by OpenMP; programmer ensures correct synchronization around shared I/O .

#### MPI Gotchas Details

- **Blocking calls:** Most MPI-1 primitives block until completion; can lead to deadlocks if not carefully ordered .

- **Data matching:** Send/recv must agree on tag, communicator, and data size/type.
  - **Collective calls:** All ranks must participate; mismatched calls cause hangs .
- 

#### 4. IaaS Cloud s Open-MPI on AWS EC2

- **IaaS Overview:** Provision virtual compute (instances), storage, networking on demand; elastic scale-up/down.
  - **AWS EC2 for MPI:**
    1. **Cluster Setup:** Launch homogeneous EC2 instances (e.g., C5 or HPC-optimized).
    2. **Networking:** Use a placement group (cluster mode) for low-latency network; configure security group to allow SSH and MPI ports.
    3. **Configuration:** Install Open MPI, set up password-less SSH among nodes (via key pairs).
    4. **Job Launch:** Use `mpirun -np N --hostfile hosts ./my_mpi_app`.
    5. **Cost Considerations:** Spot instances can reduce cost but risk preemption .
  - **Demo Links:**
    - Quick cluster on EC2:  
<https://glennklockwood.blogspot.com/2013/04/quick-mpi-cluster-setup-on-amazon-ec2.html> .
    - Java WS demo on EC2 (from Lecture 6) applies similarly for hosting microservices under EC2.
- 

#### Exam-Focus Points

- **Definitions:** SOA, REST, HTTP verbs/status codes, multiplexing, header compression, Open MPI, OpenMP, collective vs. point-to-point.
- **Use Cases:** When to choose SOAP WS-\* (complex transactions) vs. REST (simple CRUD); when to use MPI (multi-node HPC) vs. OpenMP (intra-node parallel loops).
- **Code Analysis:**



- **OpenMP:** Explain loop execution under `#pragma omp parallel for schedule(guided,4)`—how chunks are assigned and thread IDs.
- **MPI:** Trace an `MPI_Send/MPI_Recv` pair—how ranks coordinate, what happens if tags mismatch.
- **HTTP/2:** Given a network trace, identify frames (HEADERS, DATA, PUSH\_PROMISE).
- **Configuration/Deployment:** Recall EC2 instance types, security group rules, SSH key setup for MPI clusters.

## Lecture 10: JMS (Java Message Service) s Apache Kafka

---

### 1. JMS Overview

- **Definition:** JMS is the standard Java API for **Message-Oriented Middleware** (MOM) in JEE, providing a uniform way to send, receive, and process messages asynchronously or synchronously between distributed components .
- **Messaging Domains:**
  - **Point-to-Point (Queues):** One producer → one consumer. Messages sent to a Queue are consumed by a single receiver; others cannot consume the same message .
  - **Publish-Subscribe (Topics):** One producer → many subscribers. Each active subscriber gets its own copy of each message published to a Topic .
- **Consumption Modes:**
  - **Synchronous:** Client calls `MessageConsumer.receive()`, which may block until a message arrives.
  - **Asynchronous:** Client registers a `MessageListener.onMessage(Message)` callback; the JMS provider invokes it when messages arrive .
- **Durable vs. Non-Durable Subscriptions:**
  - **Non-Durable:** Subscriber only receives messages while connected.
  - **Durable:** Broker retains messages for a subscriber even when it's offline, delivering them upon reconnection .

---

### 2. JMS Architecture s Administered Objects

#### 1. Connection Factory

- Administered via JNDI; used by clients to create Connections to the JMS provider.

#### 2. Destination Administered Objects

- JMS "logical" objects (Queue or Topic) bound in JNDI, decoupling client code from physical destination names.
- Lookup via JNDI, then cast to Queue or Topic.

### 3. JMSContext (JMS 2.0+)

- Simplifies APIs: no explicit Connection, Session, or MessageProducer/Consumer objects required—uses try-with-resources.

### 4. Message Types

- TextMessage, ObjectMessage, BytesMessage, etc.

### 5. Request-Reply Pattern

- Though not a first-class model, JMS supports it via:
  1. Creating a temporary destination (Session.createTemporaryQueue()).
  2. Setting Message.setJMSReplyTo(tempDest).
  3. Consumer reads JMSReplyTo and sends the response there .

---

### 3. JMS Basic Send/Receive Code

```
// Lookup administered objects
```

```
Context ctx = new InitialContext();
```

```
ConnectionFactory cf = (ConnectionFactory)ctx.lookup("jms/CF");
```

```
Queue queue = (Queue)ctx.lookup("jms/OrderQueue");
```

```
// Produce a message (JMS 2.0 style)
```

```
try (JMSContext jctx = cf.createContext();
```

```
 JMSProducer producer = jctx.createProducer()) {
```

```
 TextMessage msg = jctx.createTextMessage("New Order: 12345");
```

```
 producer.send(queue, msg);
```

```
}
```

```
// Consume asynchronously
```

```
try (JMSContext jctx = cf.createContext()) {
```

```
 JMSConsumer consumer = jctx.createConsumer(queue);
```

```
 consumer.setMessageListener(m -> {
```

```

try {
 System.out.println("Received: " + ((TextMessage)m).getText());
} catch (JMSEException e) { /* handle */ }
});
// keep application running to listen...
}

```

**Gotcha:** Always handle JMSEException; ensure proper cleanup of connections/contexts.

---

#### 4. Transactions s Reliability

- **Container-Managed Transactions (CMT):** Use EJB/JTA to enlist JMS sessions in global transactions; JMS producer/consumer participates automatically .
  - **Bean-Managed Transactions (BMT):** Manually begin/commit via UserTransaction.
  - **Persistence s Acknowledgement Modes:**
    - AUTO\_ACKNOWLEDGE, CLIENT\_ACKNOWLEDGE, DUPS\_OK\_ACKNOWLEDGE, SESSION\_TRANSACTED.
  - **Quality-of-Service:**
    - **Persistent** vs. **Non-Persistent** delivery.
    - **Message Selectors:** Filter based on header properties.
- 

#### 5. Apache Kafka Overview

- **Definition:** A **distributed, high-throughput, fault-tolerant** message bus that decouples producers and consumers via **topics, partitions,** and **replication** .
- **Core Concepts:**
  - **Broker:** Kafka server node.
  - **Topic:** Named stream of records; split into **partitions**.
  - **Partition:** Ordered, immutable sequence of messages; each has a sequential **offset**.
  - **Producer/Consumer API:**

- **Producer** writes to topics; can choose partitioning strategy (round-robin, key-based).
- **Consumer** pulls messages; tracks offsets; can join a **consumer group** for load balancing and fault tolerance.
- **Durability s Scalability:**
  - Messages are **persisted to disk** and **replicated** across multiple brokers.
  - Handles **back-pressure**: slow consumers do not block producers.
- **Ecosystem:**
  - **Kafka Connect**: Pluggable connectors for integrating external systems (databases, HDFS) without custom code.
  - **Kafka Streams**: Lightweight library for building stream-processing applications within the broker cluster.

---

## 6. JMS vs. Kafka: Key Differences s Use Cases

Aspect	JMS (MOM)	Kafka
<b>Model</b>	Broker-mediated, push to consumer	Pull-based, log-structured storage
<b>Scalability</b>	Limited by broker; harder to scale horizontally	Highly scalable via partitioning and replication
<b>Durability</b>	Optional persistence per message	Persistent by default; configurable retention policies
<b>Ordering</b>	Per-destination	Guaranteed per-partition
<b>Consumer Semantics</b>	At-most-once (default), transacted = exactly-once	Exactly-once (with idempotent producers + transactions)
<b>API Complexity</b>	Standard Java API, more configuration overhead	Simple producer/consumer model; rich ecosystem

**Gotcha:** With Kafka, **schema evolution** and **consumer rebalancing** can introduce complexity—use the **Schema Registry** and tune **heartbeat/timeout** settings appropriately.

---

## 7. Exam-Focus Points

- **Definitions:** MOM, JMS (Queue vs. Topic), message models (sync vs. async), JMSReplyTo, broker, topic, partition, offset.
  - **Architecture:** Role of ConnectionFactory, Destinations, JMSContext, Broker vs. Consumer group in Kafka.
  - **Code Analysis:**
    - Trace a JMS send/receive sequence: JNDI lookup → JMSContext → producer/consumer → MessageListener.
    - Explain what happens on consumer.poll() in Kafka: network fetch → record batch → offset commit.
  - **Use Cases:**
    - JMS for **transactional, JEE-integrated** messaging (e.g., order processing).
    - Kafka for **high-volume, event streaming**, log aggregation, real-time analytics.
  - **Minor Details:**
    - JMS message selectors, acknowledgement modes, durable subscription lifecycle.
    - Kafka's retention policies, log compaction, exactly-once delivery guarantees.
-

## Lecture 11: EJB 3.x s Akka Actors

---

### Part I – EJB 3.x

#### 1. EJB Overview s Container

- **Enterprise Java Beans (EJB)** are managed components running in a **container** that provides services like transactions, security, pooling and remote access .
- Beans interact with the container via:
  - **JNDI/ENC** lookups for resources (JDBC, other beans).
  - **Callback methods** (@PostConstruct, @PreDestroy, etc.) invoked at lifecycle events .
  - **EJBContext** (SessionContext or EntityContext) to query runtime info (caller identity, transaction status).

#### 2. Bean Types in EJB 3.x

##### 1. Session Beans

- **Stateless**
  - No client-specific state between invocations.
  - Ideal for **POJO-like** service facades.
  - Container may pool instances; methods must be reentrant.
- **Stateful**
  - Maintains conversational state across multiple method calls for a single client.
  - Useful for shopping carts, wizards.
  - Must handle passivation/activation (serialization to disk when idle).

##### 2. Message-Driven Beans (MDBs)

- React to JMS messages asynchronously.
- Pooled like stateless beans; beware out-of-order delivery .

##### 3. Entity Beans (now via JPA)

- Represent persisted data. In EJB 3.x, use **JPA annotations** (@Entity, @Id, etc.) instead of home/remote interfaces .

- **CMP** (Container-Managed Persistence): container maps fields to DB automatically.
- **BMP** (Bean-Managed Persistence): bean code (e.g. JDBC) handles its own persistence.

### 3. EJB 3.x vs. EJB 2.x

- **POJO-style** with annotations (no verbose XML).
- **Dependency Injection** (@EJB, @Resource) replaces manual JNDI lookups.
- **No home/remote interfaces** for local beans; remote interfaces still extend javax.ejb.Remote.
- **Simpler packaging**: metadata in @ annotations or minimal ejb-jar.xml.

### 4. Session Bean Sample (Stateless)

@Stateless

```
public class HotelClerkBean implements HotelClerk {
```

```
 @PersistenceContext
```

```
 private EntityManager em;
```

```
 public Reservation book(Guest g, Room r) {
```

```
 Reservation res = new Reservation(g, r);
```

```
 em.persist(res);
```

```
 return res;
```

```
 }
```

```
}
```

- **Annotations**: @Stateless, @PersistenceContext replace XML descriptors .
- **Container-managed transactions** by default (REQUIRED), so book(...) runs in a transaction without manual begin/commit.

### 5. Bean Lifecycle s Gotchas

- **Stateless**:
  - @PostConstruct → business methods → @PreDestroy.
- **Stateful**:



- @PostConstruct → methods → idle ⇒ **passivation** (@PrePassivate) → **activation** (@PostActivate) → @PreDestroy.
  - **MDB:**
    - @PostConstruct → on message delivery via onMessage(...) → @PreDestroy; container may invoke multiple instances concurrently—design for **idempotence** .
  - **Transactions:**
    - **Container-managed** is recommended; bean-managed (UserTransaction) allows manual control but adds complexity.
  - **Remote Access:**
    - Under the hood uses **RMI/IIOP**; remote interfaces extend EJBObject and are proxies/stubs .
- 

## Part II – Akka Actors

### 1. Actors Model Fundamentals

- **Actor** is the primitive unit of computation:
  1. **Encapsulates state** (no shared mutable state).
  2. **Processes messages** asynchronously via its mailbox.
  3. **Can** create other actors, send messages, and change behavior.
- **Each actor** has a unique **ActorRef**—all interactions are via message passing.

### 2. Actor System s Addressing

- **ActorSystem**: top-level container for actors; manages threading, dispatchers.
- **Paths s Addresses**:
  - Local: system.actorOf(Props.create(MyActor.class), "myActor").
  - Remote: configured via akka.remote.netty.tcp settings; actors referenced as akka://sys@host:port/user/myActor.

### 3. Remote Deployment s Clustering

- **Remote Deployment**: specify in config that certain actor props run on remote nodes.

- **Clustering:** use **Akka Cluster** to partition work, achieve fault-tolerance; cluster membership managed dynamically.

#### 4. Reactive s Event-Driven

- **Non-blocking, event-driven** model well-suited for high-concurrency/microservices.
- Fault-handling via **supervision:** parent actors decide restart/stop strategy for children on failure.

#### 5. Comparison: EJB vs. Akka

Aspect	EJB Session Beans	Akka Actors
<b>State</b>	Container-managed or client-tied	Encapsulated per actor (in-memory)
<b>Concurrency</b>	Synchronous method calls	Asynchronous message dispatch
<b>Scaling</b>	Vertical (app-server clustering)	Horizontal (actor distribution + cluster)
<b>Fault-tolerance</b>	Container-managed transactions	Supervisor hierarchies
<b>Use Cases</b>	Transactional business logic	Event-driven, streaming, IoT, real-time

---

#### Exam-Focus Points

- **Definitions:** Stateless vs. stateful session beans; MDB; CMP vs. BMP; actor; mailbox; supervision.
- **Life-Cycle Callbacks:** Know when @PostConstruct, @PreDestroy, @PrePassivate, @PostActivate, onMessage occur.
- **Code Analysis:** Given an annotated bean or actor, explain flows (injection, transaction, message handling).
- **Gotchas:**
  - MDBs may process messages out of order—design for idempotence .
  - Stateful beans must handle passivation; avoid holding large state.
  - Actors share no state—communication only via immutable messages.

Yes — Lecture 12 is the final chapter.

## Lecture 12: Distributed Architectures s Protocols Overview

---

### 1. Architectural Styles

- **Client–Server**

- Traditional two-tier model: clients make requests, server handles both presentation and data logic.
- **Gotcha**: tight coupling; scaling the server may become a bottleneck.

- **n-Tiers (typically 3-Tier)**

1. **Presentation** (UI)

2. **Logic** (business rules)

3. **Data** (database)

- Linear topology eases separation of concerns.
- **MVC vs. 3-Tier**: MVC's triangular topology has the View directly influenced by the Model (e.g., Spring MVC, JSF) .
- **MVP** pattern (e.g. Java Swing, ASP.NET MVP) further decouples UI from logic via a "Presenter."

- **Enterprise Service Bus (ESB)**

- Middleware backbone for connecting heterogeneous services.
- Provides routing, transformation, and orchestration.
- **Gotcha**: can introduce a single point of failure or performance bottleneck if not clustered.

- **Service-Oriented Architectures s Microservices**

- Decompose applications into fine-grained services (often over HTTP/REST or SOAP).
- **Gotcha**: network latency, eventual consistency, and increased operational complexity.

---

### 2. Peer-to-Peer (P2P)

- **Pure P2P**

- All nodes are equal, share resources directly (e.g., BitTorrent, early JXTA) .
  - **Gotcha:** NAT traversal, security/trust, and discovery can be challenging.
  - **Hybrid P2P**
    - Super-peer or tracker-based (e.g., BitTorrent's tracker).
    - Balances decentralization with some central coordination.
  - **Use Cases**
    - File sharing (Torrent), blockchain networks (Bitcoin, Ethereum).
    - **Minor Detail:** replication factor and incentive mechanisms (e.g., tit-for-tat in BitTorrent).
- 

### 3. Grid s HPC Systems

- **Grid Middleware**
    - **Globus Toolkit, Condor, gLite/OGSA/OGSI, ZeroC ICE GRID, BOINC .**
    - Federated clusters across administrative domains.
  - **Key Characteristics**
    - Resource heterogeneity, wide-area networking, security via GSI (Grid Security Infrastructure).
    - **Gotcha:** job scheduling fairness, data staging and replica catalog management.
  - **Hybrid: MapReduce**
    - Master–worker model (e.g., Apache Hadoop).
    - Emphasizes data locality and fault tolerance via re-execution of failed tasks.
    - **Minor Detail:** combiners to reduce shuffle volume; speculative execution to handle stragglers.
- 

### 4. Cloud Service Models

- **IaaS** (Infrastructure as a Service)
  - Virtual machines, storage, networking (e.g., AWS EC2).

- **CaaS** (Container as a Service)
    - Container orchestration platforms (e.g., Kubernetes).
  - **PaaS** (Platform as a Service)
    - Managed runtime environments (e.g., Google App Engine).
  - **SaaS** (Software as a Service)
    - Fully delivered applications (e.g., Salesforce).
  - **Gotcha**: vendor lock-in, cost management, and multi-tenant security concerns.
- 

## 5. Communication Protocols s Platforms

- **Transport s Socket APIs**
  - **TCP/IP vs. UDP/IP**: reliable vs. best-effort delivery; choice impacts latency vs. throughput.
  - Raw sockets vs. higher-level libraries.
- **RPC s Middleware**
  - **MP-TCP, Sun RPC, gRPC** (HTTP/2-based RPC).
  - **CORBA/DCOM**: language-agnostic ORBs with rich interface definition languages.
  - **JRMI**: Java's RPC over JRMP (covered in Lecture 6).
- **Message-Oriented Middleware**
  - **JMS/ActiveMQ, RabbitMQ, Kafka** (covered in Lecture 10).
  - Patterns: queues vs. topics; push vs. pull; idempotence and exactly-once semantics.
- **Web Services s REST**
  - **SOAP/WS-\*** vs. **REST/JSON** (covered in Lecture 8).
  - **Gotcha**: SOAP adds enterprise QoS but at the cost of verbosity; REST leverages HTTP semantics but needs careful hypermedia design.
- **P2P Protocols**
  - **JXTA** (now deprecated), **Bitcoin/Ethereum** peer protocols, **BitTorrent**.

- **Minor Detail:** DHTs (distributed hash tables) for peer discovery (e.g., Kademlia).
  - **Agents and Actor Systems**
    - **JADE** (Java Agent DEvelopment Framework) for FIPA-compliant agents.
    - **Akka Actors** for high-concurrency, message-driven designs (covered in Lecture 11).
- 

## 6. Exam-Focus Points

- **Definitions:** Be ready to contrast Client–Server vs. n-Tier vs. P2P vs. Grid vs. Cloud.
  - **Use Cases:** Which architecture fits file sharing vs. HPC vs. enterprise integration vs. event streaming.
  - **Protocol Trade-offs:** Reliability vs. latency (TCP vs. UDP), tight vs. loose coupling (RPC vs. MOM).
  - **Code/Configuration:** MapReduce job lifecycle, setting up an ESB route, or initializing a P2P node.
  - **Gotchas:**
    - MVC vs. 3-Tier coupling nuances.
    - Grid security and data staging pitfalls.
    - Over- or under-partitioning in P2P DHTs.
    - Cloud cost and multi-tenant security.
- 
- 
-

# Partea II:

## 1. Java RMI (Remote Method Invocation)

### Ce este Java RMI?

- Permite invocarea de metode de la distanță (pe un alt JVM).
- Seamănă cu apelarea unui obiect local, dar are loc între mașini virtuale diferite.

### Arhitectură:

- **RMI Registry** – "nume și adresă" pentru obiectele partajate.
- **Clientul** caută în Registry un obiect (lookup).
- **Stub** – trimite parametrii spre server (marshalling).
- **Skeleton** – primește datele (unmarshalling), execută metoda și trimite răspunsul înapoi.

### Pași dezvoltare:

#### Server:

1. Definire interfață Remote cu metode ce pot fi apelate.
2. Clasă care implementează interfața + extinde UnicastRemoteObject.
3. Main Server – set SecurityManager, creează obiectul, îl înregistrează în Registry (Naming.rebind).
4. Compilezi + rmic pt Stub & Skeleton.

**Client:**

5. Set SecurityManager.
6. Obține obiectul remote cu Naming.lookup("rmi://host:1099/NAME").
7. Invocă metode ca pe un obiect local.

**Exemplu URL:** rmi://localhost:1099/SAMPLE-SERVER

**✅ Fișier de securitate (java.policy):**

Exemplu:

java

Copiază codul

```
grant {
 permission java.io.FilePermission "d:/tmp", "read,write";
 permission java.net.SocketPermission "*:1024-65535", "connect,accept";
};
```

---

**📌 2. I/O & Middleware****✅ Java I/O – concepte:**

- **InputStream / OutputStream** – byte-level
- **Reader / Writer** – char-level
- **Serialization** – persistarea obiectelor Java:
  - Serializable permite salvarea obiectelor.
  - transient → câmpuri ce nu sunt salvate.

---

**📌 3. JNI (Java Native Interface)**

Permite apelul de cod nativ (C/C++) din Java.

**✅ Pași:**



1. Definește metode native în Java.
2. Compilezi și generezi .h cu javah.
3. Scrii implementarea în C/C++.
4. Compilezi .dll / .so (lib).
5. În Java, încarci librăria: `System.loadLibrary("nume")`.

---

#### 4. REST vs SOAP (WS-\*)

##### SOAP (WS-\*) – greoi, XML, WSDL

xml

Copiază codul

```
<soap:Envelope>
 <soap:Body>
 <GetUserDetails>
 <UserID>12345</UserID>
 </GetUserDetails>
 </soap:Body>
</soap:Envelope>
```

##### REST – ușor, HTTP+JSON sau XML

http

Copiază codul

GET /users/12345 HTTP/1.1

Host: example.com

Accept: application/json

## ✅ Principii REST:

- **CRUD = HTTP Methods**

- GET – Read
- POST – Create
- PUT – Update
- DELETE – Delete

- **URL-uri semnificative:**

- /reservations/premier
- /reservations/regular

---

## 📌 5. HTTP/2 vs HTTP/1

### ✅ Îmbunătățiri HTTP/2:

- Binary (vs text)
- Multiplexing (paralelism pe o conexiune)
- Header compression (HPACK)
- Server push
- ALPN – conexiuni criptate mai rapide

 HTTP/1 făcea blocking; HTTP/2 e paralel, mai rapid și mai eficient.

---

## 6. OpenMP (Shared Memory – Paralelism pe CPU)

### Concepte:

- Paralelism pe fire de execuție (threads), partajează memorie.
- API: directive compilator (#pragma omp), biblioteci, variabile de mediu.

### Modelul Fork-Join:

- Un thread master → creează multiple threads → se unesc din nou la final.

### Directiva principală:

C

Copiază codul

```
#pragma omp parallel
{
 int id = omp_get_thread_num();
 printf("Thread ID = %d\n", id);
}
```

### Scopul variabilelor:

- private, shared, firstprivate, default(none)
- reduction(+:sum) pentru agregări sigure

### Construcții utile:

- #pragma omp for – paralelizare loop

- `schedule(static|dynamic|guided)` – modul de împărțire a iterărilor

---

## 7. OpenMPI (Distributed Memory – Comunicare între procese)

### Diferențe față de OpenMP:

Caracteristică	OpenMP	OpenMPI
Model	Shared memory	Distributed memory
Unități	Threads	Procese independente
Comunicare	Memorie comună	Mesaje (send/receive)
Scop	Pe o mașină	Pe mai multe noduri
Ușor de folosit	Mai simplu pentru 1 host	Necesită setup mai complex

### Tipuri de comunicare MPI:

- Blocking vs Non-blocking
- Point-to-point: `MPI_Send`, `MPI_Recv`
- Collective: `MPI_Bcast`, `MPI_Reduce`

---

## 8. Concepte Cloud: IaaS / PaaS / SaaS

Nivel	Exemplu	Ce oferă
IaaS	AWS EC2	Infrastructură (VM, storage)
PaaS	Google App Engine	Platformă (runtime, DB, etc.)
SaaS	Google Docs, Gmail	Software complet gata de utilizat

---

### Concluzie de învățare rapidă:

- **Java RMI**: ştii paşii (interfaţă, implementare, client, registry).
- **REST > SOAP** pentru simplitate.
- **HTTP/2** aduce viteză şi multiplexare.
- **OpenMP** – pe un sistem, **OpenMPI** – între sisteme.
- **Cloud** – IaaS (infrastructură), PaaS (platformă), SaaS (aplicaţie).

---

---

---

# Partea III:

## ✅ I. OpenMP & MPI în C++

---

### ◆ Ce este OpenMP?

**OpenMP (Open Multi-Processing)** este un API care permite programarea paralelă pe sisteme cu memorie partajată, folosind *multi-threading*. Este utilizat în principal în C/C++ și Fortran.

---

### ◆ Cum funcționează OpenMP?

- Este bazat pe **directive de compiler** (#pragma) care sunt ușor de adăugat în codul existent.
  - Fiecare thread are acces la aceeași memorie (shared memory model).
  - Util pentru paralelizarea buclelor, secțiunilor și a altor regiuni de cod.
- 

### ◆ Exemplu simplu OpenMP – paralelizarea unei bucle

cpp

Copiază codul

```
#include <iostream>
```

```
#include <omp.h>
```

```
using namespace std;
```

```
int main() {
```

```
 int n = 10;
```

```
 int a[n];
```

```
 #pragma omp parallel for
```

```
 for (int i = 0; i < n; ++i) {
```

```
 a[i] = i * i;
```

```
 cout << "Thread " << omp_get_thread_num() << " a[" << i << "] = " << a[i] << endl;
```

```
 }
```

```
 return 0;
}
```

### ✅ Explicație:

- `#pragma omp parallel for` paralelizează bucla `for`.
  - `omp_get_thread_num()` returnează ID-ul threadului curent.
- 

### ◆ Directive utile în OpenMP:

Directivă	Scop
<code>#pragma omp parallel</code>	Creează o regiune paralelă
<code>#pragma omp for</code>	Distribuie iterațiile unei bucle <code>for</code>
<code>#pragma omp sections</code>	Împarte codul în secțiuni executate paralel
<code>#pragma omp critical</code>	Secțiune de cod care trebuie executată de un singur thread simultan
<code>#pragma omp single</code>	Cod executat de un singur thread

---

### ◆ Controlul variabilelor:

`cpp`

Copiază codul

```
#pragma omp parallel for private(i) shared(sum)
```

- `private(i)` – fiecare thread are propriul `i`.
  - `shared(sum)` – toți thread-urile accesează aceeași `sum`.
- 

### ◆ Ce este MPI?

**MPI (Message Passing Interface)** este un standard pentru comunicare între procese într-un sistem distribuit sau multi-core cu **memorie distribuită**.

---

### ◆ Cum funcționează MPI?

- Fiecare proces are propria memorie – comunicarea se face explicit prin mesaje.
- Ideal pentru aplicații distribuite, de exemplu, pe clustere.

---

### ◆ Cod minim MPI:

cpp

Copiază codul

```
#include <mpi.h>
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main(int argc, char** argv) {
```

```
 MPI_Init(&argc, &argv); // inițializare MPI
```

```
 int rank, size;
```

```
 MPI_Comm_rank(MPI_COMM_WORLD, &rank); // ID proces
```

```
 MPI_Comm_size(MPI_COMM_WORLD, &size); // număr total procese
```

```
 cout << "Hello from process " << rank << " of " << size << endl;
```

```
 MPI_Finalize(); // terminare MPI
```

```
 return 0;
```

```
}
```

---

### ◆ Comunicare între procese în MPI

cpp

Copiază codul

```
if (rank == 0) {
```

```
 int value = 100;
```

```
 MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

```
} else if (rank == 1) {
```

```
 int received;
```

```
 MPI_Recv(&received, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
 MPI_STATUS_IGNORE);
```



```
cout << "Procesul 1 a primit: " << received << endl;
}
```

---

### ◆ **Funcții principale MPI:**

<b>Funcție</b>	<b>Scop</b>
MPI_Init	Inițializează MPI
MPI_Comm_rank	Află ID-ul procesului curent
MPI_Comm_size	Află numărul total de procese
MPI_Send	Trimite un mesaj
MPI_Recv	Primește un mesaj
MPI_Bcast	Difuzează un mesaj tuturor proceselor
MPI_Reduce	Agregă date din toate procesele

---

## ✅ **II. Middleware & Remote Procedures în Java**

---

### ◆ **Middleware – Ce este?**

**Middleware-ul** este un software care face legătura între aplicații distribuite, oferind servicii precum:

- **Comunicare între procese (IPC)**
  - **Servicii de rețea**
  - **Gestionarea sesiunilor**
  - **Securitate**
- 

### ◆ **Exemple de middleware:**

- **Java RMI (Remote Method Invocation)**
  - **CORBA**
  - **Web Services (SOAP, REST)**
  - **Message Brokers (RabbitMQ, Kafka)**
- 

### ◆ **Remote Procedures în Java – RMI**

**Java RMI** permite unui obiect Java să apeleze metode pe un alt obiect aflat într-un JVM diferit (la distanță).

---

### ◆ Arhitectura Java RMI

1. **Interfața Remote** – definește metodele apelabile la distanță
  2. **Clasa Server** – implementează interfața și este înregistrată în RMI Registry
  3. **Clasa Client** – apelează metodele remote prin lookup
- 

### ◆ Exemplu complet RMI

#### 1. Interfața Hello.java

java

Copiază codul

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;
```

```
public interface Hello extends Remote {
```

```
 String sayHello(String name) throws RemoteException;
```

```
}
```

---

#### 2. Serverul HelloImpl.java

java

Copiază codul

```
import java.rmi.server.UnicastRemoteObject;
```

```
import java.rmi.RemoteException;
```

```
public class HelloImpl extends UnicastRemoteObject implements Hello {
```

```
 public HelloImpl() throws RemoteException {}
```

```
 public String sayHello(String name) throws RemoteException {
```

```
 return "Hello, " + name;
```

```
}
```

```
}
```

---

### 3. Lansare Server Server.java

java

Copiază codul

```
import java.rmi.Naming;
```

```
public class Server {
 public static void main(String[] args) {
 try {
 Hello obj = new HelloImpl();
 Naming.rebind("rmi://localhost/HelloService", obj);
 System.out.println("Server is running...");
 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}
```

---

### 4. Clientul Client.java

java

Copiază codul

```
import java.rmi.Naming;
```

```
public class Client {
 public static void main(String[] args) {
 try {
 Hello obj = (Hello) Naming.lookup("rmi://localhost/HelloService");
 System.out.println(obj.sayHello("Angelica"));
 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}
```

```
}
}
}
```

---

### ◆ Comenzi pentru rulare:

bash

Copiază codul

javac \*.java

rmiregistry & # pornește registry-ul

java Server

java Client

---

### ◆ Avantajele RMI:

- Transparență în apelurile la distanță (ca apeluri locale)
- Suportă obiecte complexe (nu doar date primitive)
- Ușor de integrat în aplicații Java

---

### ◆ Alte alternative middleware:

#### Tehnologie Descriere

**CORBA** Cross-platform, limbaj-agnostic

**SOAP** Protocol bazat pe XML

**REST** API web simplu, bazat pe HTTP

**gRPC** Protocol rapid de apeluri remote (Google)

**JMS** Mesagerie asincronă în Java